

Tema 1 – Almacenamiento de la Información

IES Ntra. Sra. de los Remedios de Ubrique

1º DAW – Bases de Datos · J. Carlos Moreno · Curso 2026/2027

\newpage

Contenido

- Tema 1 – Almacenamiento de la Información
 - Contenido
 - Objetivos del Tema
 - 1.1 INTRODUCCIÓN
 - 1.1.1 Situaciones cotidianas donde se usan bases de datos
 - 1.1.2 El volumen de datos actual
 - 1.1.3 Estructura del tema
 - 1.2 ALMACENAMIENTO DE LA INFORMACIÓN
 - 1.2.1 Almacenamiento primario
 - 1.2.2 Almacenamiento secundario
 - Estructura de un disco duro
 - Transferencia de información
 - Tipos de acceso
 - 1.2.3 Almacenamiento intermedio
 - 1.3 SOPORTES DE ALMACENAMIENTO
 - 1.3.1 Soporte de acceso directo
 - 1.3.2 Soporte de acceso secuencial
 - 1.4 SISTEMAS DE ARCHIVOS
 - 1.4.1 Registros
 - Ejemplo: Registro de alumno
 - Tipos de datos de los campos
 - 1.4.2 Archivos
 - Tipos de archivos según longitud de registros
 - Operaciones básicas sobre ficheros
 - 1.4.3 Organización Primaria de Archivos
 - 1.4.3.1 Organización secuencial
 - 1.4.3.2 Organización Relativa
 - 1.4.3.3 Dispersión o hashing
 - 1.4.4 Métodos de Acceso
 - Tipos de índices
 - 1.4.4.1 Índices Primarios
 - 1.4.4.2 Índices de Agrupamiento
 - 1.4.4.3 Índices Secundarios
 - 1.4.4.4 Otros tipos de índice
 - 1.4.4.4.1 Índices multinivel
 - 1.4.4.4.2 Árboles B y B+

- 1.4.4.4.3 Índices hash
 - 1.4.4.4.4 Índices lógicos
- 1.5 SISTEMAS DE BASES DE DATOS
 - 1.5.1 Problemas de los sistemas de ficheros
 - 1.5.2 Solución: Las Bases de Datos
 - 1.5.3 Arquitectura de Sistemas de Bases de Datos
 - Los tres niveles
 - Independencia de datos
 - 1.5.4 Modelos de Datos
 - 1.5.5 Tipos de Modelos
 - 1.5.5.1 Modelos conceptuales
 - 1.5.5.2 Modelos lógicos tradicionales
 - 1.5.5.2.1 Modelo relacional
 - 1.5.5.2.2 Modelo de red
 - 1.5.5.2.3 Modelo jerárquico
 - 1.5.5.3 Modelos lógicos avanzados
 - 1.5.5.3.1 Modelos de datos orientados a objetos
 - 1.5.5.3.2 Modelos de datos declarativos
- 1.6 SISTEMAS GESTORES DE BASES DE DATOS
 - 1.6.1 Definición y objetivos
 - Objetivos de un SGBD
 - 1.6.2 Funciones del Sistema Gestor de Base de Datos (SGBD)
 - 1.6.2.1 Catálogo (Diccionario de datos)
 - 1.6.2.2 Garantizar la integridad
 - 1.6.2.3 Permitir actualizaciones
 - 1.6.2.4 Recuperación de datos
 - 1.6.2.5 Integración
 - 1.6.2.6 Cumplir restricciones
 - 1.6.2.7 Herramientas de administración
 - 1.6.3 Componentes del SGBD
 - 1.6.3.1 Lenguajes de datos
 - 1.6.3.2 Diccionario de datos
 - 1.6.3.3 Módulos del SGBD
 - 1.6.3.4 Otros componentes
 - 1.6.4 Usuarios de los SGBD
 - Administradores
 - Diseñadores de la base de datos
 - Programadores
 - Usuarios finales
 - 1.6.5 Modelo ANSI/X3/SPARC
 - 1.6.6 Tipos de SGBD
 - Según el modelo lógico
 - Según el número de usuarios
 - Según el número de sitios
 - Según el ámbito de aplicación
 - Según los tipos de datos

- Según los lenguajes soportados
- 1.6.7 Sistemas Gestores de Bases de Datos Comerciales y Libres
 - Comparativa de SGBD
 - Versiones gratuitas vs libres
 - Factores a considerar
- 1.7 Bases de Datos Centralizadas y Distribuidas
 - 1.7.1 Ventajas de las bases de datos distribuidas
 - 1.7.2 Componentes adicionales de un DDBMS
 - 1.7.3 Arquitectura de un DDBMS
 - Parámetros de las arquitecturas
 - Autonomía
 - Distribución
 - Heterogeneidad
 - 1.7.4 Transparencia en la distribución
 - 1.7.5 Técnicas de Fragmentación, Replicación y Distribución
 - Fragmentación
 - Replicación
- GLOSARIO DE TÉRMINOS
- EJERCICIOS DE AUTOEVALUACIÓN
 - Preguntas de respuesta corta
 - Ejercicios prácticos
- RESUMEN DEL TEMA
 - Puntos clave
 - Conceptos fundamentales a recordar

Objetivos del Tema

- Conocer características principales y tipos de ficheros.
- Entender el origen de las bases de datos como alternativa a los sistemas de ficheros.
- Describir los componentes y funciones principales de un Sistema Gestor de Bases de Datos (SGBD).
- Conocer las diferencias entre SGBD libres y comerciales.
- Introducir los conceptos relacionados con bases de datos distribuidas.

1.1 INTRODUCCIÓN

¿Te has preguntado alguna vez dónde y de qué manera se almacenan y gestionan los datos que utilizamos diariamente? Si pensamos en cualquier acción de nuestra vida cotidiana, o si analizamos la mayoría de los ámbitos de actividad, nos encontramos que la utilización de las bases de datos está ampliamente extendida. Éstas, y los datos contenidos en ellas, serán imprescindibles para llevar a cabo multitud de acciones.

1.1.1 Situaciones cotidianas donde se usan bases de datos

¿Crees que no es para tanto? Piensa en las siguientes situaciones:

Situación	Tipo de datos gestionados
Seleccionar canal favorito en la TDT	Preferencias de usuario, parrilla de programación

Situación	Tipo de datos gestionados
Usar la agenda del móvil para llamar	Contactos, números de teléfono, historial de llamadas
Operar en el cajero automático	Cuentas, saldos, movimientos, tarjetas
Solicitar un certificado en un organismo público	Datos personales, registros administrativos
Acudir a la consulta del médico	Historial clínico, recetas, citas
Inscribirse en un curso o plataforma Online	Datos del alumno, matrículas, calificaciones
Usar un GPS	Mapas, rutas, tráfico en tiempo real
Reservar localidades para un evento	Butacas disponibles, precios, datos del comprador
Consumir ocio digital (streaming, videojuegos)	Catálogos, puntuaciones, perfiles de usuario
Consultar información en Internet	Bibliotecas digitales, enciclopedias, museos virtuales
Registrarse en redes sociales o foros	Perfiles, publicaciones, mensajes, relaciones sociales
Conducir un coche moderno	Navegación, diagnóstico, mantenimiento

Reflexión: Suponemos que no es necesario continuar más para darnos cuenta de que casi todo lo que nos rodea, en alguna medida, está relacionado con los datos, su almacenamiento y su gestión.

1.1.2 El volumen de datos actual

El gran volumen de datos que actualmente manejamos y sus innumerables posibilidades requieren de la existencia de técnicos perfectamente formados y capaces de trabajar con ellos. Este módulo profesional se centra en el estudio de las Bases de Datos y su uso en el desarrollo de aplicaciones.

1.1.3 Estructura del tema

En esta primera unidad comenzaremos conociendo los primeros sistemas basados en ficheros para el almacenamiento y gestión de la información. Seguidamente, se desarrollarán los conceptos y definiciones básicas relacionadas con las bases de datos, posteriormente analizaremos sus modelos y tipos, un poco más adelante, podremos conocer las características y capacidades de los sistemas gestores de bases de datos y finalmente, identificaremos las herramientas reales con las que llevar a cabo la gestión de dichas bases.

1.2 ALMACENAMIENTO DE LA INFORMACIÓN

Todas las aplicaciones informáticas trabajan en última instancia con datos o información que deben ser almacenados en un medio físico, como discos duros, memorias flash o DvD. Estos medios forman una **jerarquía** que distingue entre tres niveles de almacenamiento: **primario**, **secundario** e **intermedio**.

1.2.1 Almacenamiento primario

Se refiere a aquellos medios sobre los que la CPU del ordenador puede acceder directamente y, por tanto, más rápidamente.

Característica	Descripción
Velocidad	Muy alta (nanosegundos)
Capacidad	Limitada (GB)
Volatilidad	Volátil (se pierde al apagar)
Coste	Alto por GB
Ejemplos	Memoria RAM, caché L1/L2/L3

Componentes:

- **Memoria principal (RAM):** Almacena los programas y datos que la CPU está utilizando en cada momento.
- **Memorias caché:** Pequeñas memorias ultrarrápidas (L1, L2, L3) que almacenan copias de los datos más frecuentemente accedidos para acelerar el procesamiento.

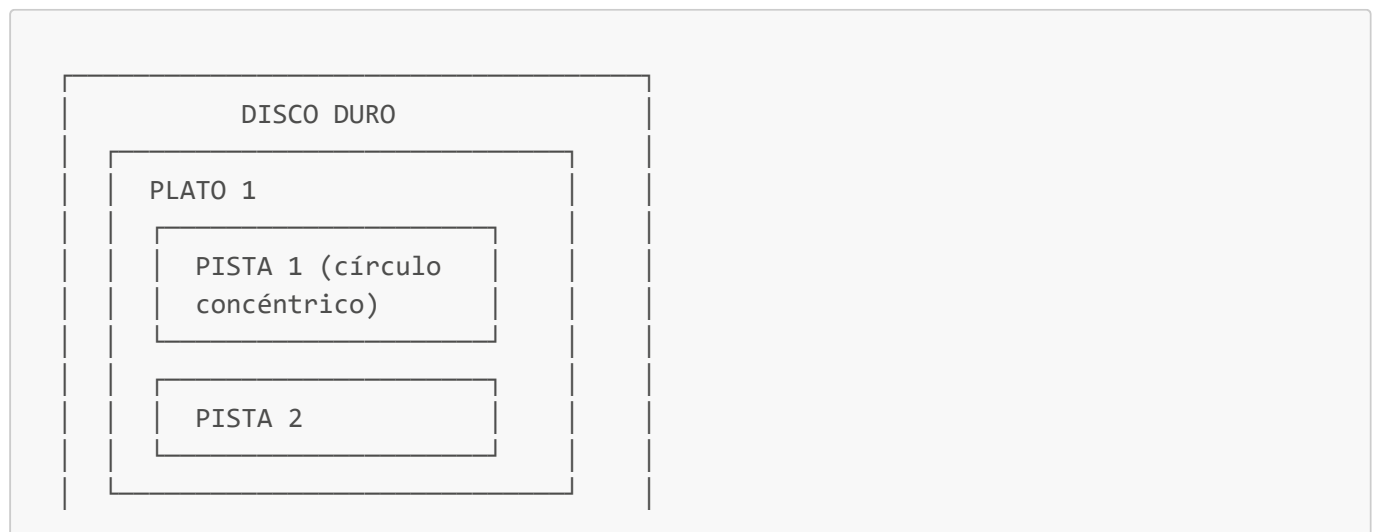
1.2.2 Almacenamiento secundario

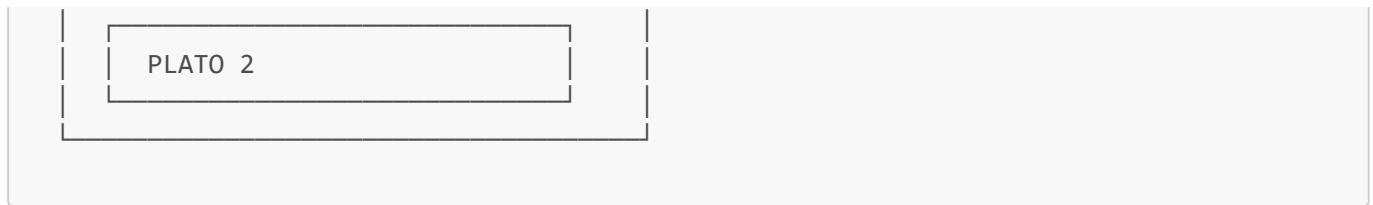
Se refiere a dispositivos más lentos, pero de mayor capacidad, como los discos ópticos y magnéticos o las cintas. Para acceder a los datos la CPU debe copiarlos previamente en el almacenamiento primario.

Característica	Descripción
Velocidad	Media-baja (milisegundos)
Capacidad	Alta (TB)
Volatilidad	No volátil (persiste sin energía)
Coste	Bajo por GB
Ejemplos	HDD, SSD, DVD, Blu-ray, cintas

Estructura de un disco duro

En un disco duro normalmente se agrupan varios **platos**, cada uno de los cuales se divide en **pistas** o círculos concéntricos. En ellas se almacena la información.





- **Cilindro:** Agrupación de pistas de todos los platos que se encuentran a la misma distancia del centro. Es importante que los datos a los que se suelen acceder simultáneamente estén en el mismo cilindro ya que se leen con mayor rapidez.
- **Bloque/Sector:** Cada pista se subdivide en bloques de un tamaño fijo, determinado por el sistema operativo al inicializarlo con un sistema de archivos dado. Los bloques, también llamados páginas, suelen tener un tamaño entre **512 y 4.096 bytes**.

Transferencia de información

La transferencia de información entre memoria y disco tiene lugar en **unidades de bloque**:

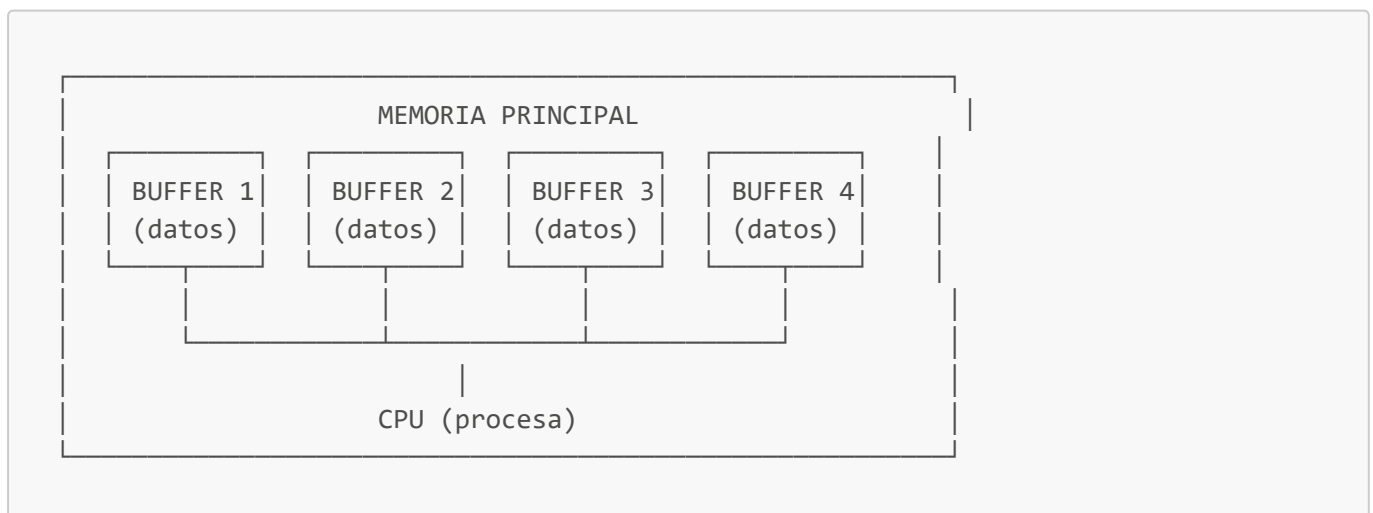
- Cuando se produce una orden de **lectura**, se copia uno o varios bloques en el llamado **buffer** de la memoria (área reservada de la memoria principal).
- Si se necesita efectuar una **escritura**, se copia el bloque correspondiente al bloque del disco.

Tipos de acceso

Tipo	Descripción	Ejemplo
Acceso aleatorio	Se puede acceder a cualquier bloque conociendo su dirección física, sin necesidad de recorrerlos todos	Discos duros, SSD, memorias USB
Acceso secuencial	Los bloques se almacenan de manera contigua y para acceder a uno de ellos necesitamos leer todos los anteriores	Cintas magnéticas

1.2.3 Almacenamiento intermedio

Cuando se necesita transferir varios bloques de disco a memoria principal y se conocen todas las direcciones de bloque, es posible reservar varias áreas de **almacenamiento intermedio** o **buffers** dentro de la memoria principal para agilizar la transferencia.



Ventaja: Mientras la CPU procesa datos de un buffer, puede leer o escribir en otro. El uso de este tipo de almacenamiento es muy común en sistemas de bases de datos.

1.3 SOPORTES DE ALMACENAMIENTO

Se distinguen dos tipos de soportes para el almacenamiento de los datos:

1.3.1 Soporte de acceso directo

Son los más empleados y el acceso a los datos puede hacerse de forma directa, pudiendo colocarnos en la posición que nos interesa y leer a partir de ella.

Soporte	Capacidad típica	Velocidad	Uso principal
Discos magnéticos (HDD)	1-20 TB	Media	Almacenamiento masivo
Discos SSD	256 GB - 4 TB	Alta	Sistema operativo, aplicaciones
Pendrive USB	32 GB - 1 TB	Media	Portabilidad
Tarjetas memoria flash	16 GB - 512 GB	Alta	Cámaras, móviles
CD/DVD	700 MB - 4.7 GB	Baja	Distribución de software
Blu-ray	25 GB - 100 GB	Media	Vídeo alta definición

1.3.2 Soporte de acceso secuencial

Se suelen usar en **copias de seguridad** y si deseamos leer un dato que está en la mitad de la cinta, tendremos que leer todo lo que hay hasta llegar a esa posición.

Soporte	Capacidad típica	Velocidad	Uso principal
Cintas magnéticas (LTO)	12-48 TB	Baja	Copias de seguridad empresariales

Nota: Las cintas magnéticas se usan sobre todo para copias de seguridad por su estabilidad, capacidad y durabilidad a largo plazo.

1.4 SISTEMAS DE ARCHIVOS

En esta sección estudiaremos someramente los conceptos relacionados con archivos tanto en lo relativo a su contenido como a su organización lógica y física.

1.4.1 Registros

La información se almacena en forma de **registros**, que son colecciones de valores o elementos de información relacionados, cada uno de los cuales corresponde a un **campo** del registro.

Ejemplo: Registro de alumno

REGISTRO DE ALUMNO			
nombre	fecha_nac	telefono	edad
"María García López"	2005-03-15	612345678	18

Tipos de datos de los campos

Cada campo tiene un **tipo de dato** que especifica el tipo de valores que puede tomar. Estos tipos suelen corresponderse con los de los lenguajes de programación:

Campo	Tipo de dato	Tamaño
nombre	CHAR/VARCHAR	50 bytes
fecha_nac	DATE	10 bytes
telefono	VARCHAR	15 bytes
edad	INT	4 bytes

Además, cada campo tiene un **tamaño** determinado en bytes que puede ser:

- **Fijo:** Todos los registros ocupan el mismo espacio.
- **Variable:** Depende del valor almacenado en cada registro.

Definición: La unión de estos campos y sus tipos determina el **tipo o formato del registro**.

1.4.2 Archivos

Podemos definir un **fichero informático** como un conjunto de registros, grabados sobre un soporte que pueda ser leído por el ordenador.

Tipos de archivos según longitud de registros

Tipo	Descripción	Ventajas	Desventajas
Longitud fija	Todos los registros son iguales en tamaño	Fáciles de manipular, acceso directo por posición	Desperdician espacio en disco
Longitud variable	Los registros son de distinto tamaño o tienen campos variables	Mejor aprovechamiento del espacio	Programas más complejos, registros no en posiciones fijas

Operaciones básicas sobre ficheros

Todos los datos son, en última instancia, gestionados mediante ficheros mediante **cuatro operaciones básicas**:

OPERACIONES BÁSICAS SOBRE FICHEROS
1. CONSULTA / LECTURA (Read) Obtener datos del fichero
2. INSERCIÓN (Insert) Añadir nuevos registros
3. MODIFICACIÓN (Update) Cambiar valores de registros
4. BORRADO (Delete) Eliminar registros

Importante: Cualquier operación más compleja (como búsqueda, ordenación, etc.) es combinación de dos o más operaciones básicas.

1.4.3 Organización Primaria de Archivos

El término **organización de ficheros** se aplica a la forma en que se colocan los datos contenidos en los registros de cada fichero sobre el soporte informático (disco, cinta, etc.) durante su grabación.

Existen **dos formas básicas** de organización de ficheros:

Organización	Descripción
Secuencial	Los registros se van grabando unos a continuación de los otros, en el orden que se van dando de alta
Relativa	Los registros se graban en las posiciones que les corresponda según el valor que guarden en el campo denominado clave

1.4.3.1 Organización secuencial

Es el tipo más básico de organización. Los registros se colocan secuencialmente uno a continuación del otro y los registros nuevos se añaden al final del fichero.

FICHERO SECUENCIAL :

Reg 1	Reg 2	Reg 3	Reg 4	Reg 5	Reg 6
(Ana)	(Luis)	(María)	(Pedro)	(Sara)	(Juan)

↑
Se lee en orden

↑
Se añade al final

Operaciones:

Operación	Eficiencia	Descripción
Inserción	★ ★ ★ Muy eficiente	Siempre se hace en el último bloque disponible
Búsqueda	★ Baja	Requiere búsqueda lineal, bloque por bloque
Borrado	★ Muy ineficiente	Quedan huecos en los bloques que no se reutilizan
Modificación	★ ★ Media	Implica búsqueda y reescritura

Problemas del borrado:

- Al eliminar registros y no ocupar el espacio liberado, quedan **huecos** en los bloques.
- Salvo que se elimine el fichero, estos huecos no serán reutilizados.
- Aunque hay técnicas para aprovechar estos espacios, lo mejor es **reorganizar el fichero periódicamente**, de modo que se empaqueten los registros eliminando los borrados.

Cálculo de acceso a registros (longitud fija):

Para el caso de archivos de registros de longitud fija, el acceso a un registro por su posición dentro del archivo es muy sencillo:

$$\text{FBL (Factor de Bloqueo)} = \text{Registro Físico} / \text{Registro Lógico}$$

$$\text{BLOQUE} = \text{parte_entera}(\text{registro} / \text{FBL})$$

$$\text{POSICIÓN dentro del bloque} = \text{registro MOD FBL}$$

Ejemplo:

- Fichero F con registro lógico de **1028 bytes**
- Sistema con registro físico de **24 KB** (24576 bytes)

$$\text{FBL} = 24576 / 1028 = 23,91 \rightarrow 23 \text{ registros lógicos por bloque}$$

Para acceder al registro 1234:

$$\text{BLOQUE} = 1234 / 23 = 53,65 \rightarrow \text{bloque } 54$$

$$\text{POSICIÓN} = 1234 \text{ MOD } 23 = 15$$

Resultado: El registro 1234 se encuentra en el bloque 54, posición 15 dentro del bloque.

Variantes de la organización secuencial:

Variante	Descripción
Secuencial indexada	Los registros se graban secuencialmente, pero se pueden recuperar con acceso directo gracias a un fichero adicional llamado índice
Secuencial encadenada	Permite tener los registros ordenados según un orden lógico diferente del orden físico mediante punteros

1.4.3.2 Organización Relativa

En este tipo de archivos los registros se graban en orden según el valor de uno de sus campos llamado **campo de ordenación**. Normalmente se usa un campo especial denominado **campo clave**, cuyos valores son distintos para cada registro.

FICHERO RELATIVO (ordenado por campo clave):

Reg 1 (001) (Ana)	Reg 2 (002) (Luis)	Reg 3 (003) (María)	Reg 4 (004) (Pedro)	Reg 5 (005) (Sara)	Reg 6 (006) (Juan)
-------------------------	--------------------------	---------------------------	---------------------------	--------------------------	--------------------------

↑
Búsqueda binaria rápida
(si busca por campo clave)

↑
Inserción costosa
(debe mantener orden)

Operaciones:

Operación	Eficiencia	Descripción
Lectura en orden	★★★★ Muy eficiente	El siguiente registro está a continuación del actual
Búsqueda por campo clave	★★★★ Muy rápida	Se puede usar búsqueda binaria
Búsqueda aleatoria	★ Baja	No hay ventajas si no se busca por campo de ordenación
Inserción	★ Costosa	Debe mantenerse el orden, puede implicar desplazamiento
Eliminación	★★ Media	Menos costosa si se usan registros marcados
Modificación	★★ Media	No afecta si no cambia el campo de ordenación

1.4.3.3 Dispersión o hashing

En este sistema se elige un campo llamado **campo de dispersión**. Al valor de ese campo se le aplica una función llamada **función de aleatorización** o **función de dispersión** que, tomando como entrada dicho valor, devuelve un número que será la **dirección del bloque de disco** en que se almacenará el registro.

FUNCIÓN DE DISPERSIÓN (HASH)

Valor del campo clave $\longrightarrow$ [Función Hash] $\longrightarrow$ Dirección

Ejemplo: "María García" $\longrightarrow$ hash() $\longrightarrow$ Bloque 42

Características:

Aspecto	Descripción
Uso principal	Acelerar el acceso a registros cuando se busca un único registro según el campo de dispersión
Campo de dispersión	Normalmente es el campo clave
Colisiones	Ocurre cuando dos valores distintos producen el mismo valor de dispersión

Soluciones a colisiones:

Técnica	Descripción
Segunda función	Usar una función hash alternativa para los registros que colisionan
Búsqueda lineal	Buscar la siguiente posición vacía dentro del bloque
Encadenamiento	Enlazar los registros que colisionan en una lista enlazada

1.4.4 Métodos de Acceso

El **método de acceso** se refiere al procedimiento seguido para acceder a uno o más registros determinados de un fichero. Una de las operaciones más costosas y frecuentes es la **búsqueda de información**, por lo cual se usan sistemas que permiten mejorar su eficiencia.

Estos sistemas se basan en el uso de **índices**, que son estructuras de datos que relacionan valores de un campo (normalmente el campo clave) de un registro con su dirección de memoria.

Analogía: Son similares a los índices analíticos de cualquier libro de texto: una lista en orden alfabético de los términos de un libro incluyendo la página o páginas en que se encuentra.

Tipos de índices

Tipo	Campo de indización	Densidad	Descripción
Primario	Campo clave de ordenación	No denso	Una entrada por bloque
Agrupamiento	Campo no clave de ordenación	No denso	Una entrada por valor distinto

Tipo	Campo de indización	Densidad	Descripción
Secundario (clave)	Campo clave no de ordenación	Denso	Una entrada por registro
Secundario (no clave)	Campo no clave	Denso/No denso	Una entrada por registro o valor

1.4.4.1 Índices Primarios

Un **índice primario** es un archivo ordenado cuyos registros, de longitud fija, tienen dos campos:

1. El **campo clave de ordenamiento** del fichero de datos.
2. Un **apuntador** a un bloque de disco.

ÍNDICE PRIMARIO:

id_jugador (campo clave)	Bloque (apuntador)
1	1
23	2
45	3
67	4
...	...

↓

Apunta al primer registro de cada bloque

Características:

- Cada entrada de índice apunta a un **grupo de registros** (bloque) del fichero de datos.
- No todos los identificadores aparecen en el índice, sino **solamente los primeros de cada bloque**.
- Se consideran **no densos**: cada entrada se asocia con varios registros de datos.
- Los ficheros de índices primarios son **mucho más pequeños** que los ficheros de datos.

1.4.4.2 Índices de Agrupamiento

Cuando los registros de un archivo están ordenados físicamente según un **campo no clave** (no tiene un valor distinto para cada registro de datos), este campo se denomina **campo de agrupamiento**.

Podemos crear un índice sobre este campo llamado **índice de agrupamiento** para acelerar la obtención de registros con el mismo valor en dicho campo.

ÍNDICE DE AGRUPAMIENTO (por num_equipo):

num_equipo (agrupamiento)	Bloque (apuntador)

1	1
2	3
3	5
...	...

↓

Una entrada por cada valor distinto del campo

Características:

- Hay una entrada de índice por cada **valor distinto** del campo de agrupamiento.
- Contiene un apuntador al **primer bloque** con ese valor en el campo.
- Normalmente, se reservan bloques para cada valor distinto del campo de agrupamiento para facilitar las inserciones.
- Si se precisa más de un bloque para un mismo valor, se van enlazando bloques adicionales mediante apuntadores de bloque.

1.4.4.3 Índices Secundarios

Son también archivos ordenados con dos campos:

1. El primero es del mismo tipo que alguno de los campos clave distintos del ordenamiento del archivo de datos.
2. El segundo es un apuntador a bloque.

Puede haber **varios índices secundarios** sobre varios campos de un mismo archivo de datos.

Tipo	Campo	Densidad	Entradas
Secundario sobre clave	Campo clave (no de ordenación)	Denso	Una por registro
Secundario sobre no clave	Campo no clave	Denso/No denso	Una por registro o valor

Nota: En el caso de índice secundario sobre campo clave, hay una entrada de índice por cada registro de datos, con lo cual tenemos un **índice denso**.

1.4.4.4 Otros tipos de índice

Existen estructuras de índice más complejas que quedan fuera del alcance de este libro. A modo de ilustración comentaremos algunos de ellos.

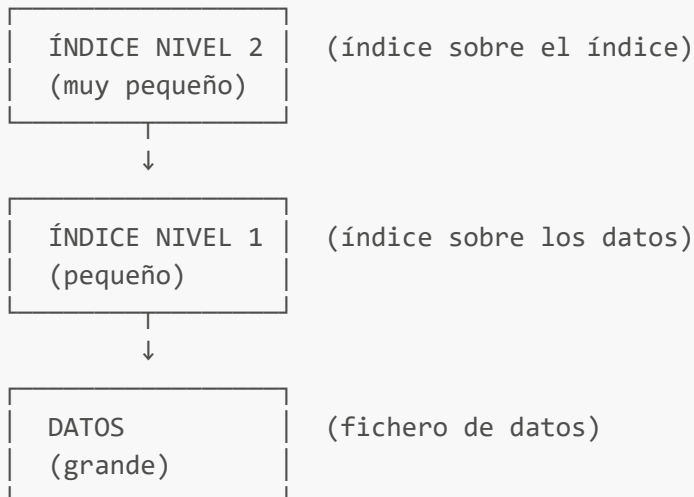
1.4.4.4.1 Índices multinivel

Como hemos visto, una de las principales ventajas de los índices es la mejora sustancial de las búsquedas de registros de datos. Esta mejora se debe, principalmente, al hecho de que estén ordenados y se pueda hacer una **búsqueda binaria**.

Estas búsquedas tienen un coste de $\log_2(b)$ accesos a bloque para localizar un registro (donde b es el número de bloques del índice).

Sin embargo, se puede mejorar todavía más restringiendo el número de bloques de índice a leer usando para ello un archivo de índice sobre el fichero de índices inicial, creando así uno de **dos niveles**. Esto se puede extender a más niveles dando lugar a **índices multinivel**.

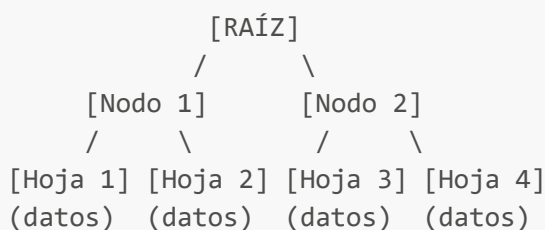
ÍNDICE MULTINIVEL :



Fórmula: En índices multinivel, el número de accesos es de $\log_n(b)$, donde n es el factor de bloques (número de registros por bloque) del índice.

1.4.4.4.2 Árboles B y B+

Son estructuras de árbol muy utilizadas en programación y en índices de bases de datos.



Características:

- Un **árbol** es un fichero en el que hay unos **nodos** (padres) que apuntan a otros (hijos).
- Los nodos hijo no pueden apuntar a nodos padre.
- Se forma una estructura con un **nodo raíz** sin padre cuyos nodos hijos son padres de otros hasta llegar a los **nodos hoja** (sin hijos).
- Estos nodos almacenan apuntadores a nodos hijo, apuntadores a registros de datos y valores de campos de los registros de datos.
- La estructura que forman hace muy eficiente la búsqueda de datos.

1.4.4.4.3 Índices hash

Es posible crear estructuras de acceso de tipo índice usando técnicas de dispersión vistas en la sección anterior.

En este caso, cada entrada de índice se organiza como un **archivo de dispersión** y contiene registros formados por el valor de dispersión y un apuntador al bloque de datos.

Así, el acceso a una de ellas requiere aplicar la función hash correspondiente al valor buscado y obtener así el valor del apuntador.

1.4.4.4.4 Índices lógicos

Hasta ahora hemos supuesto que las entradas de índice contienen un valor lógico (el valor del campo) y un valor físico, o apuntador, que especifica la dirección física del registro o bloque de datos.

Esto tiene la desventaja de que si los registros de datos cambian con frecuencia de lugar físico, debe actualizarse el índice en consonancia, lo que puede resultar muy costoso en términos computacionales.

Para solucionar esto se usan **índices lógicos** en los que cada entrada está formada por:

- Un valor de indización secundario.
- El valor del campo clave, que determina la organización primaria del archivo.

De este modo, la búsqueda de un registro de datos según un valor de índice secundario permitirá obtener el valor correspondiente al campo de ordenación del archivo y realizar la búsqueda directamente en el archivo de datos.

1.5 SISTEMAS DE BASES DE DATOS

Inicialmente, cuando las primeras empresas y organizaciones empezaron a usar sistemas informáticos trabajaban con **sistemas de ficheros**. Es decir, se trabajaba con programas que manejaban información almacenada en ficheros.

Cada equipo trabajaba con sus propios datos y programas y se encargaba de su mantenimiento y gestión. Al principio el sistema funcionó, pero con el tiempo y, sobre todo, con el incremento de la cantidad de información así como de los usuarios que la manejaban, surgieron problemas.

1.5.1 Problemas de los sistemas de ficheros

Problema	Descripción
Separación y aislamiento de los datos	Cuando los datos se separan en distintos ficheros es más complicado acceder a ellos, ya que el programador de aplicaciones debe sincronizar el procesamiento de los distintos ficheros implicados
Duplicación de datos	La redundancia de datos existente en los sistemas de ficheros hace que se desperdicie espacio de almacenamiento y puede llevar a que se pierda la consistencia de los datos

Problema	Descripción
Dependencia de datos	La estructura física de los datos se encuentra codificada en los programas de aplicación, cualquier cambio en dicha estructura es difícil de realizar
Formatos de ficheros incompatibles	La estructura de los ficheros se define en los programas de aplicación, es completamente dependiente del lenguaje de programación
Consultas fijas y proliferación de programas	Cualquier consulta o informe que se quiera realizar debe ser programado por el programador de aplicaciones
Control de concurrencia	El acceso de varios clientes al mismo fichero genera inconsistencias
Autorizaciones	Los errores en los permisos de ficheros pueden hacer que un mismo cliente pueda modificar un dato en un fichero y no en otro
Catálogo	Resulta complicado saber dónde están los distintos datos. No existe un esquema general que muestre la organización de la información

1.5.2 Solución: Las Bases de Datos

Para trabajar de un modo más efectivo, surgieron las **bases de datos** como modelo de organización de los datos y, con ellas, los **sistemas de gestión de bases de datos (SGBD)** como herramienta que permite la implementación y gestión de bases de datos.

Definición: Una **base de datos** es un conjunto de datos almacenados entre los que existen relaciones lógicas y ha sido diseñada para satisfacer los requerimientos de información de una empresa u organización.

Características principales:

Característica	Descripción
Centralización	Los datos están centralizados y organizados, de forma que se minimice la redundancia
Compartición	La base de datos no pertenece a un equipo, se comparte por toda la organización
Metadatos	La base de datos no solo contiene los datos de la organización, también almacena una descripción de dichos datos
Diccionario de datos	La descripción de los datos se almacena en el diccionario de datos o catálogo que, en muchos casos, se organiza en otra base de datos

1.5.3 Arquitectura de Sistemas de Bases de Datos

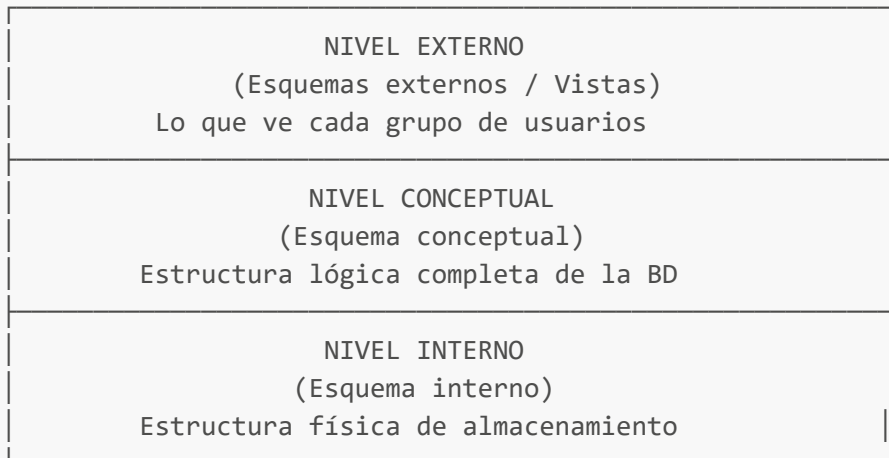
En 1975, el comité **ANSI-SPARC** (American National Standard Institute - Standards Planning and Requirements Committee) propuso un estándar para la creación de sistemas de bases de datos basado en una **arquitectura de tres niveles**, que resulta muy útil a la hora de conseguir estas tres características:

- **Independencia lógica**
- **Independencia física**

- **Integridad y seguridad**

El objetivo de la arquitectura de tres niveles es el de separar en niveles de abstracción el esquema de una base de datos. Son tres formas distintas de ver o representar una misma base de datos.

Los tres niveles



Nivel	Esquema	Descripción
Interno	Esquema interno	Describe la estructura física de la base de datos: ficheros, discos, directorios, métodos de acceso
Conceptual	Esquema conceptual	Describe la estructura de toda la base de datos para una comunidad de usuarios: entidades, atributos, relaciones, restricciones
Externo	Esquemas externos	Describe la parte de la base de datos que interesa a un grupo de usuarios determinados

Independencia de datos

La arquitectura de tres niveles es útil para explicar el concepto de **independencia de datos**, que podemos definir como la capacidad para modificar el esquema en un nivel del sistema sin tener que modificar el esquema del nivel inmediato superior.

Tipo	Definición	Ejemplo
Independencia lógica	Capacidad de modificar el esquema conceptual sin tener que alterar los esquemas externos ni los programas de aplicación	Eliminar una entidad que no es usada por ciertas vistas
Independencia física	Capacidad de modificar el esquema interno sin tener que alterar el esquema conceptual	Reorganizar ficheros físicos para mejorar el rendimiento

Nota: La independencia física es más fácil de conseguir que la independencia lógica.

1.5.4 Modelos de Datos

La base de datos consiste entonces en los datos concretos referentes a un sistema o parte del mundo que hemos modelado (por ejemplo, nuestra base de datos de la liga de baloncesto incluye todos los datos relativos a jugadores, equipos, partidos, etc.).

Estos datos son sencillos de manejar cuando son unos pocos, pero cuando su volumen crece se requiere el uso de distintos modelos para facilitar el diseño de las mismas.

Definición: Un **modelo de datos** es una colección de herramientas conceptuales para describir los datos, las relaciones que existen entre ellos y sus restricciones.

Un modelo nos proporciona mecanismos de abstracción para representar una parte del mundo cuyos datos nos interesan. Dicha representación, realizada en términos de un modelo dado, recibe el nombre de **esquema** y el conjunto de datos que representa es la **base de datos**.

1.5.5 Tipos de Modelos

La arquitectura de tres niveles nos obliga a modelar nuestros datos en cada nivel. En este libro nos centraremos en los dos primeros (global y externo), ya que los del nivel interno son específicos del software utilizado.

En este sentido distinguimos **tres tipos** de modelos:

Tipo	Nivel	Descripción
Conceptuales	Global	Representan los datos de forma parecida a como los captamos en el mundo real
Lógicos tradicionales	Global/Externo	Utilizan tablas de registros para representar los objetos modelados
Lógicos avanzados	Global/Externo	Modelos más recientes para aplicaciones específicas con datos complejos

1.5.5.1 Modelos conceptuales

Se usan para describir datos en el nivel global. Con este modelo representamos los datos de forma parecida a como nosotros los captamos en el mundo real.

Características:

- Capacidad de estructuración bastante flexible.
- Permiten especificar restricciones de datos explícitamente.
- El más utilizado es el modelo **Entidad-Relación (MER)**.

Elementos del MER:

Elemento	Descripción	Ejemplo
Entidad	Objeto que existe y se distingue de otros por sus características	Jugador, Equipo, Partido

Elemento	Descripción	Ejemplo
Atributo	Característica de una entidad	Nombre, Altura, Fecha de nacimiento
Relación	Asociación entre entidades	"pertenece" (jugador pertenece a equipo)

Nota: Este tipo de modelos utiliza una simbología para representar cada elemento. Lo estudiaremos en detalle en el Tema 4.

1.5.5.2 Modelos lógicos tradicionales

Fueron los primeros en usarse, aunque su uso hoy en día, especialmente el relacional, está muy extendido. Se utilizan para describir datos en el nivel global, pero de un modo más lógico (más cercano a la máquina).

Estos modelos utilizan **tablas de registros** para representar los objetos modelados y sus relaciones.

Los tres modelos de datos más ampliamente aceptados son:

Modelo	Descripción	Uso actual
Relacional	Representa los datos y las relaciones a través de una colección de tablas (filas = registros, columnas = atributos)	★ ★ ★ Muy extendido
De red	Permite que un mismo nodo tenga varios padres	★ Obsoleto
Jerárquico	Organiza los datos en una estructura de árbol	★ Obsoleto

1.5.5.2.1 Modelo relacional

En este modelo se representan los datos y las relaciones entre estos, a través de una colección de **tablas**, en las cuales:

- Las **filas** (tuplas) equivalen a cada uno de los registros que contendrá la base de datos.
- Las **columnas** corresponden a las características (atributos) de cada registro localizado en la tupla.

También sirven para representar el nivel externo (vistas) de una base de datos.

1.5.5.2.2 Modelo de red

Éste es un modelo ligeramente distinto del jerárquico; su diferencia fundamental es la modificación del concepto de nodo: se permite que un mismo nodo tenga **varios padres** (posibilidad no permitida en el modelo jerárquico).

Fue una gran mejora con respecto al modelo jerárquico, ya que ofrecía una solución eficiente al problema de redundancia de datos; pero, aun así, la dificultad que significa administrar la información en una base de datos de red ha significado que sea un modelo utilizado en su mayoría por programadores más que por usuarios finales.

1.5.5.2.3 Modelo jerárquico

Éstas son modelos de bases de datos que, como su nombre indica, almacenan su información en una estructura jerárquica. En este modelo los datos se organizan en una forma similar a un **árbol** (visto al revés), donde:

- Un **nodo padre** de información puede tener varios **hijos**.
- El nodo que no tiene padres es llamado **raíz**.
- A los nodos que no tienen hijos se les conoce como **nodos hoja**.

Las bases de datos jerárquicas son especialmente útiles en el caso de aplicaciones que manejan un gran volumen de información y datos muy compartidos, permitiendo crear estructuras estables y de gran rendimiento a la hora de acceder a los mismos.

1.5.5.3 Modelos lógicos avanzados

Son modelos de datos relativamente recientes y cada vez más utilizados, sobre todo en aplicaciones específicas que manejan nuevos y más complejos tipos de datos.

1.5.5.3.1 Modelos de datos orientados a objetos

Estos modelos son utilizados, sobre todo, en aplicaciones programadas bajo el paradigma de la **orientación a objetos**. Tratan de almacenar en la base de datos no solo los datos (estado), sino también la funcionalidad asociada (comportamiento).

Una base de datos orientada a objetos es una base de datos que incorpora todos los conceptos importantes del paradigma de objetos:

Concepto	Descripción
Encapsulación	Propiedad que permite ocultar la información al resto de los objetos, impidiendo así accesos incorrectos o conflictos
Herencia	Propiedad a través de la cual los objetos o tablas heredan atributos y métodos de otros situados en un nivel superior según una jerarquía de clases
Polimorfismo	Hace referencia a métodos que pueden aplicarse a distintos tipos de objetos

1.5.5.3.2 Modelos de datos declarativos

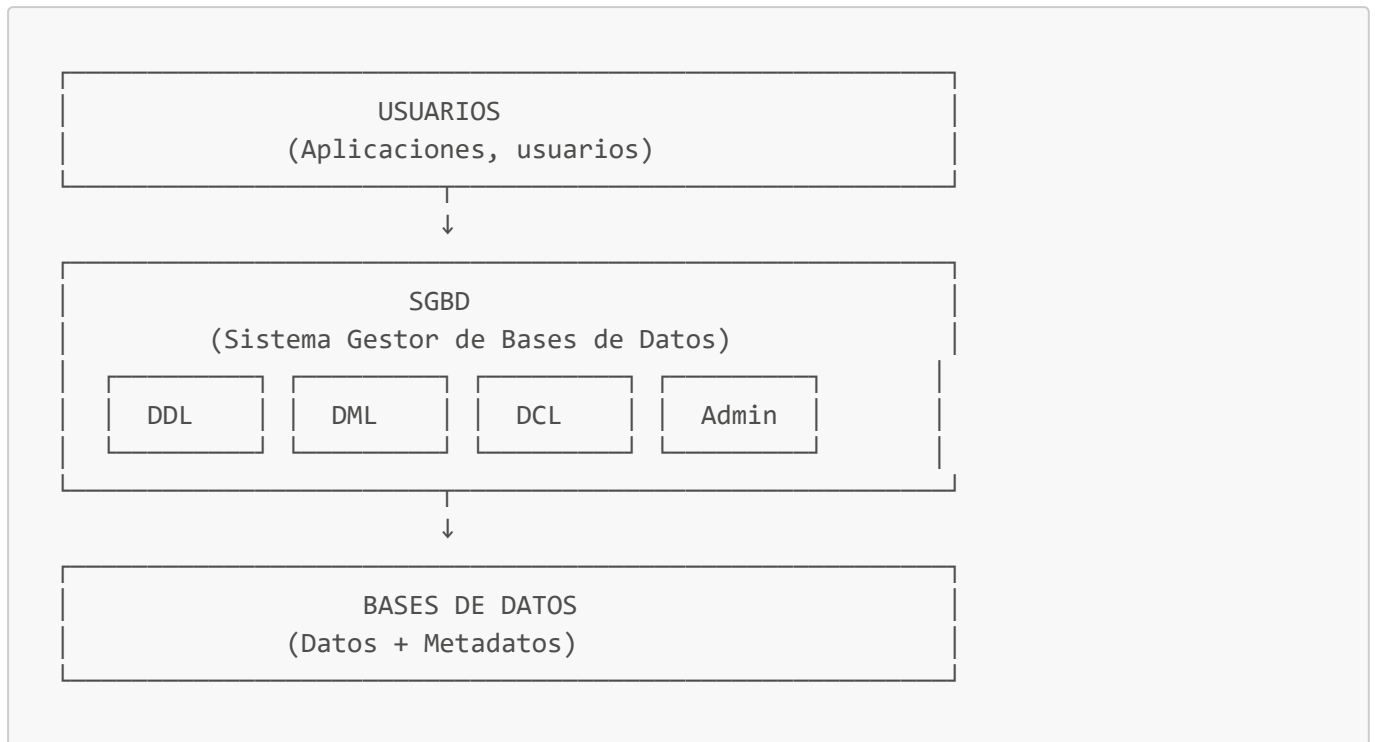
Estos modelos se dividen en **deductivos** y **funcionales**. Suelen usarse para **bases de conocimiento**, que no son más que bases de datos con mecanismos de consulta en los que el trabajo de extracción de información a partir de los datos recae en realidad sobre el sistema informático, en lugar de sobre el usuario.

1.6 SISTEMAS GESTORES DE BASES DE DATOS

Los modelos nos permiten representar nuestra información de un modo sencillo y en un lenguaje común. Sin embargo, necesitamos un **software** que nos permita llevarlo a cabo o implementar dichos modelos. Este software es lo que se denominaremos **SGBD**.

1.6.1 Definición y objetivos

Definición: El **sistema de gestión de la base de datos (SGBD)** es una aplicación que permite a los usuarios definir, crear y mantener la base de datos y proporciona acceso controlado a la misma. Es una herramienta que sirve de **interfaz** entre el usuario y las bases de datos.



Objetivos de un SGBD

Objetivo	Descripción
Asegurar los tres niveles de abstracción	Físico, lógico y externo
Permitir la independencia física y lógica de los datos	Los cambios en un nivel no afectan al nivel superior
Garantizar la consistencia de los datos	Puede haber datos duplicados o derivados que deben mantener sus valores de forma coherente
Ofrecer seguridad de acceso	Control de acceso a los datos por parte de usuarios y grupos
Gestión de transacciones	Garantizar la ejecución de un conjunto de operaciones críticas como una sola operación
Permitir la concurrencia	Acceso de varios usuarios sobre los mismos datos mediante bloqueos que mantienen la integridad

1.6.2 Funciones del Sistema Gestor de Base de Datos (SGBD)

Para la consecución de los objetivos comentados en la sección anterior, la mayoría de SGBD comerciales y libres incorporan las siguientes características y funciones:

1.6.2.1 Catálogo (Diccionario de datos)

Un **catálogo** donde se almacenen las descripciones de los datos y sea accesible por los usuarios. Este catálogo es lo que se denomina **diccionario de datos** y contiene información que describe los datos de la base de datos (metadatos).

Normalmente, un diccionario de datos describe entre otras cosas:

Información	Descripción
Nombre, tipo y tamaño de los datos	Estructura de las tablas y campos
Relaciones entre los datos	Claves primarias, foráneas, etc.
Restricciones de integridad	Reglas que deben cumplir los datos
Usuarios autorizados	Permisos de acceso a los objetos
Estadísticas de utilización	Frecuencia de transacciones, número de accesos

1.6.2.2 Garantizar la integridad

Disponer de un mecanismo que garantice que todas las actualizaciones correspondientes a una **transacción** se realicen o que no se realice ninguna.

Definición: Una **transacción** es un conjunto de acciones que cambian el contenido de la base de datos.

Ejemplo de transacción:

- Dar de alta a un jugador.
- Eliminar un inmueble.
- Eliminar un jugador y reasignar sus inmuebles a otro jugador.

Si la transacción falla durante su realización, por ejemplo, porque falla el hardware, la base de datos quedará en un estado **inconsistente**. Algunos de los cambios se habrán hecho y otros no, por tanto, los cambios realizados deberán ser **deshechos** para devolver la base de datos a un estado consistente.

1.6.2.3 Permitir actualizaciones

Asegurar que la base de datos se actualice correctamente cuando varios usuarios la están actualizando concurrentemente.

Uno de los principales objetivos de los SGBD es el de permitir que varios usuarios tengan **acceso concurrente** a los datos que comparten.

El acceso concurrente es relativamente fácil de gestionar si todos los usuarios se dedican a **leer** datos, ya que no pueden interferir unos con otros. Sin embargo, cuando dos o más usuarios están accediendo a la base de datos y al menos uno de ellos está **actualizando** datos, pueden interferir produciendo inconsistencias en la base de datos.

1.6.2.4 Recuperación de datos

Permitir recuperar las bases de datos en caso de que ocurra algún suceso imprevisto que afecte o destruya la base de datos.

Causa de fallo	Descripción
Fallo de hardware	Error en disco, memoria, etc.
Error de software	Bug en el SGBD o en el sistema operativo
Error del usuario	El usuario detecta un error durante la transacción y la aborta
Pérdida de datos	Por cualquier motivo

En todos estos casos, el SGBD debe proporcionar mecanismos capaces de **recuperar** y llevar la base de datos a un estado consistente lo más cercano posible en el tiempo al momento del fallo.

1.6.2.5 Integración

Ser capaz de integrarse con algún **software de comunicación**. Muchos usuarios acceden a la base de datos desde terminales. Algunas veces estos terminales se encuentran conectados directamente a la máquina sobre la que funciona el SGBD. En otras ocasiones, los terminales están en lugares remotos, por lo que la comunicación con la máquina que alberga al SGBD se debe hacer a través de una red.

1.6.2.6 Cumplir restricciones

Proporcionar los medios necesarios para garantizar que, tanto los datos de la base de datos como los cambios que se realizan sobre estos datos, sigan ciertas reglas.

La **integridad** se ocupa de la calidad de los datos. Normalmente se expresa mediante **restricciones**, que son una serie de reglas que la base de datos no puede violar.

Ejemplo: En una base de datos de una liga de baloncesto se puede establecer la restricción de que cada equipo no puede tener asignados más de veinte jugadores. En este caso sería deseable que el SGBD controlara que no se sobrepase este límite cada vez que se asigne un jugador a un equipo.

1.6.2.7 Herramientas de administración

Proporcionar herramientas que permitan administrar la base de datos de modo efectivo, lo que implica:

- Un diseño óptimo de las mismas.
- Garantizar la disponibilidad e integridad de los datos.
- Controlar el acceso al servidor y a los datos.
- Monitorizar el funcionamiento del servidor.
- Optimizar su funcionamiento.

Muchas de ellas van integradas en el sistema gestor, otras son creadas por terceros o por el propio administrador según sus requerimientos.

1.6.3 Componentes del SGBD

Son los elementos que deben proporcionar los servicios comentados en la sección anterior. No se pueden generalizar ya que varían mucho según la tecnología. Sin embargo, normalmente todo SGBD incluye los siguientes:

1.6.3.1 Lenguajes de datos

Son lenguajes para la manipulación de datos, tanto desde el punto de vista de su acceso y modificación como del control y seguridad de los mismos. Normalmente se distinguen tres tipos según su funcionalidad:

Lenguaje	Siglas	Función	Ejemplos
Lenguaje de Definición de Datos	DDL (Data Definition Language)	Definir y describir los objetos de la base de datos, su estructura, relaciones y restricciones	CREATE, ALTER, DROP
Lenguaje de Control de Datos	DCL (Data Control Language)	Control y seguridad de los datos (privilegios y modos de acceso)	GRANT, REVOKE
Lenguaje de Manipulación de Datos	DML (Data Manipulation Language)	Manipulación del contenido de las bases de datos	INSERT, UPDATE, DELETE, SELECT

Nota: Para todos estos lenguajes se usa principalmente el lenguaje **SQL** (Structured Query Language). Incluye instrucciones para los tres tipos de lenguajes comentados y por su sencillez y potencia se ha convertido en el lenguaje estándar de los SGBD relacionales.

1.6.3.2 Diccionario de datos

Esquemas que describen el contenido del SGBD incluyendo los distintos objetos con sus propiedades. Objetos:

- Tablas base y vistas (tablas derivadas).
- Consultas.
- Dominios y tipos definidos de datos.
- Restricciones de tabla y dominio y aserciones.
- Funciones y procedimientos almacenados.
- Disparadores o triggers.

1.6.3.3 Módulos del SGBD

Módulo	Función
Seguridad	Los usuarios no autorizados no pueden acceder a la base de datos
Integridad	Mantiene la integridad y la consistencia de los datos
Control de concurrencia	Permite el acceso compartido a la base de datos
Control de recuperación	Restablece la base de datos después de un fallo
Gestión del diccionario de datos	Accesible por el usuario, contiene la descripción de los datos

Módulo	Función
Programación de aplicaciones	Facilita el desarrollo de aplicaciones
Importación/exportación de datos	Migraciones entre sistemas
Distribución de datos	Gestión de datos distribuidos
Replicación	Arquitectura maestro-esclavo
Sincronización	De equipos replicados

1.6.3.4 Otros componentes

Componente	Función
Optimizador de consultas	Determina la estrategia óptima para la ejecución de las consultas
Gestión de transacciones	Realiza el procesamiento de las transacciones
Planificador (scheduler)	Programa y automatiza la realización de ciertas operaciones y procesos
Copias de seguridad	Garantiza que la base de datos se puede devolver a un estado consistente en caso de fallo

Nota: No todos los SGBD presentan la misma funcionalidad, depende de cada producto. En general, los grandes SGBD multiusuario ofrecen todas las funciones que se acaban de citar y muchas más.

Los sistemas modernos son conjuntos de programas extremadamente complejos y sofisticados, con millones de líneas de código y con una documentación consistente en varios volúmenes. Los SGBD están en continua evolución, tratando de satisfacer los requerimientos de todo tipo de usuarios.

1.6.4 Usuarios de los SGBD

Generalmente, distinguimos **cuatro grupos** de usuarios de sistemas gestores de bases de datos:

Tipo de usuario	Nivel de abstracción	Funciones
Administradores	Físico	Instalación, configuración, control de acceso, seguridad, monitorización y optimización del sistema
Diseñadores de la base de datos	Lógico	Diseño lógico de la base de datos: identificar datos, relaciones y restricciones
Programadores	Aplicación	Desarrollo de aplicaciones que interactúan con las bases de datos
Usuarios finales	Externo	Acceso a los datos mediante vistas, sin conocer el funcionamiento interno

Administradores

Trabajan en el nivel de abstracción **físico** relacionado con el almacenamiento. Distinguimos:

- **Administradores del sistema gestor:** Encargados de la instalación y configuración del sistema, del control de acceso a los recursos, de la seguridad y de la monitorización y optimización del sistema gestor.
- **Administradores de bases de datos:** Se encargan del diseño físico de la misma, implementación y mantenimiento de la base de datos.

Diseñadores de la base de datos

Realizan el **diseño lógico** de la base de datos, debiendo identificar los datos, las relaciones entre datos y las restricciones sobre los datos y sus relaciones.

El diseñador de la base de datos debe tener un profundo conocimiento de los datos de la empresa y también debe conocer sus **reglas de negocio**. Las reglas de negocio describen las características principales de los datos tal y como los ve la empresa.

Para obtener un buen resultado, el diseñador de la base de datos debe implicar en el desarrollo del modelo de datos a todos los usuarios de la base de datos, tan pronto como sea posible.

Importante: El diseño lógico de la base de datos es **independiente** del SGBD concreto que se vaya a utilizar, es independiente de los programas de aplicación, de los lenguajes de programación y de cualquier otra consideración física.

Programadores

Tanto de aplicaciones que, mediante **API** de lenguajes de programación interactúan con las bases de datos como de objetos de la base de datos, como rutinas almacenadas o disparadores.

Estas aplicaciones servirán a los usuarios finales para, de una forma amigable, poder consultar datos, insertarlos, actualizarlos y eliminarlos.

Usuarios finales

Trabajan en el nivel **externo** mediante vistas o porciones de las bases de datos. Son clientes de las bases de datos que hacen uso de ellas sin conocer en absoluto su funcionamiento y organización interna. Son personas con pocos o nulos conocimientos de informática.

1.6.5 Modelo ANSI/X3/SPARC

El organismo **ANSI** ha marcado la referencia para la construcción de SGBD. El modelo definido por el grupo de trabajo **SPARC** se basa en estudios anteriores en los que se definían los tres niveles de abstracción necesarios para describir una base de datos.

ANSI profundiza más en esta idea y define cómo debe ser el proceso de creación y utilización de estos niveles.

Nivel	Esquema	Descripción
Externo	Esquemas externos	Reflejan la información preparada (filtrada en forma de vistas) para el usuario final
Conceptual	Esquema conceptual	Refleja los datos y relaciones de la base de datos (el nivel lógico)
Interno	Esquema interno	Determina la organización física de los datos (sistemas de ficheros, estructura de directorios, espacios de almacenamiento, índices, etc.)

Analogía: Podemos hacer un paralelismo entre una casa y un SGBD:

- El **nivel externo** sería lo que vemos nosotros como usuarios de la casa (enchufes, luces, paredes, etc.).
- El **nivel lógico o conceptual** correspondería a los planos detallados de la casa.
- El **nivel físico** correspondería a cómo están hechas realmente las obras según el terreno, orientación y otros condicionantes físicos.

En definitiva, el modelo ANSI es una **propuesta teórica** sobre las características deseables en un sistema gestor de bases de datos.

1.6.6 Tipos de SGBD

Existen numerosos SGBD en el mercado que podemos clasificar según los siguientes criterios:

Según el modelo lógico

Modelo	Descripción
Jerárquico	Estructura de árbol
En red	Grafos arbitrarios
Relacional	Tablas (el más extendido)
Objeto-relacional	Relacional + orientación a objetos
Orientado a objetos	Objetos con datos y métodos

Según el número de usuarios

Tipo	Descripción
Monousuario	Solo permiten un usuario
Multiusuario	Permiten la conexión de varios usuarios

Según el número de sitios

Tipo	Descripción
Centralizados	En un solo servidor o equipo

Tipo	Descripción
Distribuidos	En varios equipos que pueden ser homogéneos y heterogéneos

Según el ámbito de aplicación

Tipo	Descripción
Propósito General	Orientados a toda clase de aplicaciones
Propósito Específico	Centradas en un tipo específico de aplicaciones

Según los tipos de datos

Tipo	Descripción
Sistemas relacionales estándar	Manejan tipos básicos (int, char, etc.)
XML	Para bases de datos que trabajan con documentos XML
Objeto-relacionales	Para bases relacionales que incorporan tipos complejos de datos
De objetos	Para bases de datos que soportan tipos de objeto con datos y métodos asociados

Según los lenguajes soportados

Tipo	Descripción
SQL estándar	Lenguaje estándar para SGBD relacionales
NoSQL	Menos estructurado, orientado a bases documentales o de tipo clave-valor. Muy útil para manejar consultas de grandes cantidades de datos distribuidos en clusters de servidores

Nota: Dentro de todos ellos, los que con gran diferencia se han impuesto en casi todos los ámbitos del desarrollo han sido los basados en el **modelo relacional**. Ello se ha debido principalmente a su flexibilidad y sencillez de manejo. Igualmente, conviene destacar la amplia implantación del lenguaje **SQL**, que se ha convertido en un estándar para el manejo de datos en el modelo relacional.

Cabe destacar, sin embargo, la creciente popularidad de otros lenguajes como **NoSQL** (Not Only SQL), que se han desarrollado para adaptarse a datos masivos y dispersos en muchos servidores.

1.6.7 Sistemas Gestores de Bases de Datos Comerciales y Libres

Con el advenimiento de Internet, el **software libre** se ha consolidado como alternativa, técnicamente viable y económicamente sostenible al software comercial.

Importante: Lo que comúnmente se denomina software libre **no significa que sea gratuito**, ya que muchas empresas ofrecen servicios de soporte y mantenimiento para productos (en ocasiones de pago como Red Hat o Suse) pero cuyo código sigue siendo accesible.

Comparativa de SGBD

SGBD Comercial	SGBD Libre	Observaciones
Oracle	MySQL (Oracle)	MySQL es de código abierto pero propiedad de Oracle
DB2 (IBM)	PostgreSQL	PostgreSQL es más potente que MySQL en muchos aspectos
SQL Server (Microsoft)	MariaDB	Fork de MySQL mantenido por la comunidad
Informix (IBM)	Firebird	Firebird es un derivado de InterBase de Borland
Sybase	SQLite	SQLite es ideal para aplicaciones embebidas y móviles

Versiones gratuitas vs libres

Cada vez más los fabricantes ofrecen **versiones gratuitas** (que no libres), aunque con limitaciones en su funcionalidad, de sus productos que permiten probarlos y aprender sus interioridades. Estas versiones se suelen denominar de tipo **express**.

Factores a considerar

Factor	Software Libre	Software Comercial
Coste	Generalmente gratuito	Licencias costosas
Soporte	Comunidad, documentación online	Soporte oficial garantizado
Flexibilidad	Alto control, personalizable	Más cerrado y rígido
Independencia	Independencia del proveedor	Dependencia del proveedor
Conocimiento requerido	Mayor conocimiento del producto	Más fácil de usar para principiantes
Documentación	Generada por la comunidad, siempre actualizada	Documentación oficial de calidad

Conclusión: Usar software libre o no es una elección del consumidor. Debe considerar estos y otros factores de la manera que mejor se adapte a sus necesidades. No se trata tanto de defender "qué es mejor en sí mismo", sino de tener la información y poder elegir "qué es mejor según mis circunstancias".

1.7 Bases de Datos Centralizadas y Distribuidas

En un sistema de bases de datos **centralizado** todos los componentes (software, datos y soportes físicos) residen en un único lugar físico. Los clientes (aplicaciones, funciones, programas cliente, usuarios) acceden al sistema a través de distintas interfaces que se conectan al servidor.

Sin embargo, existe otra arquitectura cada vez más extendida en la que se opta por un esquema **distribuido** en el que los componentes se distribuyen en distintos computadores comunicados a través de una red de

cómputo. Dichos sistemas se conocen con el nombre de **Sistemas Gestores de Bases de Datos Distribuidas** o **DDBMS**.

Definición: Una **base de datos distribuida** es una colección de datos que pertenece lógicamente al mismo sistema pero que se encuentra físicamente almacenada en distintas máquinas conectadas por una red.

1.7.1 Ventajas de las bases de datos distribuidas

Ventaja	Descripción
Mejora de rendimiento	Las consultas y transacciones que afectan a un solo sitio son más rápidas al ser más pequeña la base de datos local
Fiabilidad	Probabilidad de que un sistema esté activo en un tiempo dado. Si falla algún sitio es poco probable que afecte a la mayoría de transacciones
Disponibilidad	Probabilidad de que un sistema esté activo de manera continuada. Mejorada especialmente por la replicación de datos
Tipos de aplicaciones	Muchas aplicaciones tienen un marcado carácter distribuido al estar sus componentes dispersos en distintos sitios
Acceso y procesamiento más rápido	Varios nodos comparten carga de trabajo
Acceso desde cualquier ubicación	Desde una ubicación puede accederse a información alojada en diferentes lugares
Costes inferiores	Los costes son inferiores a los de las bases centralizadas
Tolerancia a fallos	Mediante la replicación, si un nodo deja de funcionar el sistema completo no deja de funcionar
Adaptación a la organización	Permiten la incorporación de nodos de forma flexible y fácil
Independencia local	Aunque los nodos están interconectados, tienen independencia local

1.7.2 Componentes adicionales de un DDBMS

Distribuir implica una mayor complejidad en el diseño, implementación y gestión de los datos, para lo cual un buen SGBDD debe incorporar, además de los componentes habituales en un SGBD centralizado:

- Tener acceso a **sitios remotos** e intercambiar información con los mismos para la ejecución de consultas y transacciones.
- Disponer de un **catálogo** con información sobre cómo están distribuidos y replicados los datos del sistema distribuido.
- Poder **optimizar** consultas y transacciones sobre datos que estén en más de un sitio.
- Mantener la **integridad** en los permisos sobre datos replicados.
- Mantener la **consistencia** de las copias de un elemento replicado.
- Garantizar la **recuperación** del sistema en una caída.

1.7.3 Arquitectura de un DDBMS

En general, en un sistema distribuido disponemos de varias formas o modelos para organizar el software. Se habla de la arquitectura **cliente-servidor** consistente en dividir el software (la funcionalidad) entre equipos que hacen de clientes y otros que hacen de servidores.

Parámetros de las arquitecturas

Parámetro	Descripción	Tipos
Autonomía	Quién tiene el control sobre qué datos del SGBDD	Integración fuerte, Sistema semiautónomo, Sistema aislado
Distribución	Cómo se distribuyen los datos a lo largo del sistema	Distribución cero, Cliente-servidor, Servidores colaborativos
Heterogeneidad	Heterogeneidad de los componentes del sistema	Hardware, Comunicaciones, Sistema operativo

Autonomía

Tipo	Descripción
Integración fuerte	Un equipo hace de coordinador enviando las solicitudes de información a los equipos donde resida la información
Sistema semiautónomo	Los SGBD son independientes pero participan en el conjunto pudiendo compartir parte de sus datos
Sistema aislado	El SGBD no tiene constancia de que existan otros gestores

Distribución

Tipo	Descripción
Distribución cero	No hay distribución de datos
Cliente-servidor	Los datos residen en los servidores mientras que los clientes proveen una interfaz de acceso a los mismos
Servidores colaborativos	No se distingue entre servidores y clientes, cada máquina tiene toda la funcionalidad del SGBDD

Heterogeneidad

Se refiere a la heterogeneidad de los componentes del sistema en distintos niveles como:

- Hardware.
- Comunicaciones.
- Sistema operativo.

Cualquier combinación de estos parámetros determina un modelo arquitectónico distinto para nuestro SGBDD.

1.7.4 Transparencia en la distribución

Un SGBDD debe proporcionar lo que se conoce como **transparencia en la distribución**, lo que implica que cuando un usuario accede a la base, todo lo que ocurre es transparente y queda al margen de cualquier detalle de cómo se ejecuta la transacción.

Es decir, al usuario se le presenta un esquema que no incluye información sobre la distribución de los datos. Es el software cliente quien consulta al catálogo del SGBDD para conocer la ubicación de los datos y poder así descomponer la consulta y distribuirla entre los distintos servidores.

Tipo de transparencia	Descripción
Transparencia de red	El usuario no necesita saber dónde están los datos
Transparencia de replicación	El usuario no necesita saber qué datos están replicados
Transparencia de fragmentación	El usuario no necesita saber qué datos están fragmentados

1.7.5 Técnicas de Fragmentación, Replicación y Distribución

Diseñar bases de datos distribuidas implica decidir sobre cómo fragmentarlas, cómo replicarlas y cómo distribuirlas, además del diseño previo de la base de datos en sí. La información sobre cómo se hace este proceso se almacena en el **catálogo global** del sistema al que tiene acceso el cliente cuando quiere acceder a la información.

Fragmentación

Consideraremos a nuestra base de datos formada por unidades lógicas que son las **relaciones** o **tablas**.

Tipo	Descripción	Ejemplo
Fragmentación horizontal	Divide las tuplas (filas) según una o varias condiciones sobre distintos atributos	Separar clientes por regiones de España
Fragmentación vertical	Divide los atributos (columnas) de una tabla	Campos ligeros (nombre, dirección) en un servidor, campos pesados (foto, historial) en otro
Fragmentación mixta	Combinación de horizontal y vertical	Combinar ambos criterios

Ejemplo de fragmentación horizontal:

Tabla CLIENTES original:

id	nombre	región

1	Ana	Andalucía
2	Luis	Madrid
3	María	Andalucía
4	Pedro	Cataluña

Fragmento 1 (Andalucía):

id	nombre
1	Ana
3	María

Fragmento 2 (Madrid):

id	nombre
2	Luis

Fragmento 3 (Cataluña):

id	nombre
4	Pedro

Ejemplo de fragmentación vertical:

Tabla CLIENTES original:

id	nombre	dirección	foto	historial
1	Ana	C/ Mayor	[datos]	[datos]

Fragmento 1 (servidor potente): Fragmento 2 (servidor almacenamiento):

id	nombre	dirección
1	Ana	C/ Mayor

id	foto	historial
1	[datos]	[datos]

↑

Se une por el atributo común (id)

↑

Importante: La fragmentación vertical debe hacerse con cuidado, ya que tenemos que incluir un **atributo común**, usualmente una **clave**, en ambos fragmentos para poder recuperar la información completa cuando sea necesario.

Nota: Debemos ser conscientes de que fragmentar es un proceso complejo que solo debe hacerse cuando se considere necesario por motivos de eficiencia, ya que al hacerlo hacemos también más complejo (más costoso en términos de CPU y consumo de recursos en general) el acceso a los datos por parte de los clientes.

Replicación

Es fundamentalmente útil para mejorar la **disponibilidad** de los datos.

Tipo	Descripción	Ventajas	Desventajas
Replicación total	La misma copia de la base de datos en todos los sitios	Máxima disponibilidad	Baja eficiencia en actualizaciones

Tipo	Descripción	Ventajas	Desventajas
No replicación	Cada fragmento está en un sitio (reparto no redundante)	Eficiente en actualizaciones	Baja disponibilidad
Replicación parcial	Réplicas de algunos fragmentos en algunos sitios	Equilibrio entre disponibilidad y eficiencia	Complejidad de gestión

Nota: La replicación, como hemos señalado, distribuye la carga, acelera consultas y mejora la disponibilidad pero, sobre todo, es óptima para sistemas con muchas lecturas. Sin embargo, requiere muchas más actualizaciones con el consiguiente consumo de almacenamiento.

GLOSARIO DE TÉRMINOS

Término	Definición
Almacenamiento primario	Medios sobre los que la CPU puede acceder directamente (RAM, caché)
Almacenamiento secundario	Dispositivos más lentos pero de mayor capacidad (discos, cintas)
Buffer	Área de memoria reservada para almacenar temporalmente datos durante la transferencia
Registro	Colección de valores o elementos de información relacionados (campos)
Campo	Elemento individual de información dentro de un registro
Fichero	Conjunto de registros grabados sobre un soporte
Organización secuencial	Los registros se graban uno a continuación del otro
Organización relativa	Los registros se graban según el valor de un campo clave
Dispersión (hashing)	Técnica que aplica una función a un campo para obtener la dirección de almacenamiento
Índice	Estructura de datos que relaciona valores de un campo con su dirección
Índice primario	Índice sobre el campo clave de ordenación
Índice de agrupamiento	Índice sobre un campo no clave de ordenación
Índice secundario	Índice sobre un campo que no es el de ordenación
Base de datos	Conjunto de datos almacenados con relaciones lógicas, diseñado para satisfacer los requerimientos de información de una organización
SGBD	Sistema Gestor de Bases de Datos: software que permite definir, crear y mantener bases de datos

Término	Definición
Modelo de datos	Colección de herramientas conceptuales para describir datos, relaciones y restricciones
MER	Modelo Entidad-Relación: modelo conceptual que representa la realidad mediante entidades, atributos y relaciones
Modelo relacional	Modelo lógico que representa los datos mediante tablas
DDL	Data Definition Language: lenguaje para definir la estructura de la base de datos
DML	Data Manipulation Language: lenguaje para manipular los datos
DCL	Data Control Language: lenguaje para controlar el acceso a los datos
SQL	Structured Query Language: lenguaje estándar para SGBD relacionales
Transacción	Conjunto de acciones que cambian el contenido de la base de datos como una unidad
Independencia de datos	Capacidad de modificar un esquema sin afectar al nivel superior
Independencia lógica	Capacidad de modificar el esquema conceptual sin afectar a los esquemas externos
Independencia física	Capacidad de modificar el esquema interno sin afectar al esquema conceptual
DDBMS	Sistema Gestor de Bases de Datos Distribuidas
Fragmentación	Proceso de dividir una base de datos en partes más pequeñas
Replicación	Proceso de crear copias de los datos en distintos sitios
Metadatos	Datos que describen otros datos (diccionario de datos)

EJERCICIOS DE AUTOEVALUACIÓN

Preguntas de respuesta corta

1. **¿Cuáles son los tres niveles de almacenamiento? Describe brevemente cada uno.**
2. **¿Qué diferencia existe entre acceso directo y acceso secuencial? Pon un ejemplo de cada uno.**
3. **¿Qué es un registro? ¿Qué es un campo? Pon un ejemplo.**
4. **¿Cuáles son las cuatro operaciones básicas que se pueden realizar sobre un fichero?**
5. **¿Qué ventajas y desventajas tiene la organización secuencial frente a la organización relativa?**
6. **¿Qué es una función de dispersión (hash)? ¿Para qué se utiliza?**
7. **¿Qué tipos de índices conoces? Describe brevemente cada uno.**

8. **¿Cuáles son los principales problemas de los sistemas de ficheros?**
9. **¿Qué es una base de datos? ¿Qué la diferencia de un sistema de ficheros?**
10. **¿Cuáles son los tres niveles de la arquitectura ANSI-SPARC?**
11. **¿Qué es la independencia de datos? ¿Qué tipos existen?**
12. **¿Qué es un SGBD? ¿Cuáles son sus funciones principales?**
13. **¿Qué lenguajes se utilizan en un SGBD? ¿Para qué sirve cada uno?**
14. **¿Qué tipos de SGBD conoces? Clasifícalos según diferentes criterios.**
15. **¿Qué ventajas tiene una base de datos distribuida frente a una centralizada?**
16. **¿Qué es la fragmentación? ¿Qué tipos existen?**
17. **¿Qué es la replicación? ¿Para qué se utiliza?**
18. **¿Qué es la transparencia en una base de datos distribuida?**
19. **¿Qué es una transacción? ¿Por qué es importante?**
20. **¿Qué es el diccionario de datos? ¿Qué información contiene?**

Ejercicios prácticos

Ejercicio 1: Calcula el factor de bloqueo (FBL) para un fichero con registros lógicos de 512 bytes y un registro físico de 8 KB. ¿En qué bloque se encuentra el registro número 100? ¿Qué posición ocupa dentro de ese bloque?

Ejercicio 2: Diseña un registro para almacenar información de los libros de una biblioteca. Incluye al menos 5 campos con sus tipos de datos y tamaños.

Ejercicio 3: Explica con tus palabras la diferencia entre independencia lógica e independencia física. Pon un ejemplo de cada una.

Ejercicio 4: Dada una tabla de empleados con los campos (id, nombre, departamento, salario), diseña: a) Un índice primario sobre el campo id. b) Un índice de agrupamiento sobre el campo departamento. c) Un índice secundario sobre el campo nombre.

Ejercicio 5: Investiga qué SGBD comerciales y libres conoces. Crea una tabla comparativa con al menos 4 de cada tipo, indicando sus principales características.

RESUMEN DEL TEMA

En este tema hemos estudiado los fundamentos del almacenamiento de la información y la evolución desde los sistemas de ficheros hasta las bases de datos modernas.

Puntos clave

1. **Almacenamiento:** Existen tres niveles (primario, secundario e intermedio) con diferentes características de velocidad, capacidad y volatilidad.
2. **Sistemas de ficheros:** La información se organiza en registros (campos) y ficheros (conjuntos de registros). Pueden organizarse de forma secuencial, relativa o mediante dispersión (hashing).
3. **Métodos de acceso:** Los índices (primarios, de agrupamiento y secundarios) permiten acelerar las búsquedas sobre los ficheros.
4. **Problemas de los ficheros:** La redundancia, la falta de independencia de datos, las dificultades de concurrencia y la falta de flexibilidad en las consultas llevaron al desarrollo de las bases de datos.
5. **Bases de datos:** Conjuntos de datos organizados que minimizan la redundancia y facilitan la gestión. Se basan en modelos de datos (conceptuales, lógicos tradicionales y avanzados).
6. **SGBD:** Software que permite definir, crear y mantener bases de datos. Incluye lenguajes (DDL, DML, DCL), diccionario de datos, y módulos de seguridad, integridad, concurrencia y recuperación.
7. **Arquitectura ANSI-SPARC:** Tres niveles de abstracción (externo, conceptual, interno) que permiten la independencia de datos.
8. **Tipos de SGBD:** Pueden clasificarse según el modelo lógico, número de usuarios, número de sitios, ámbito de aplicación, tipos de datos y lenguajes soportados.
9. **Bases de datos distribuidas:** Permiten distribuir los datos en distintos sitios, mejorando el rendimiento, fiabilidad y disponibilidad, aunque aumentando la complejidad.

Conceptos fundamentales a recordar

- **Registro:** Colección de campos relacionados.
- **Fichero:** Conjunto de registros.
- **Índice:** Estructura que acelera las búsquedas.
- **Base de datos:** Conjunto de datos organizados con relaciones lógicas.
- **SGBD:** Software de gestión de bases de datos.
- **Independencia de datos:** Capacidad de modificar un nivel sin afectar al superior.
- **Transacción:** Conjunto de operaciones que se ejecutan como una unidad.
- **Fragmentación:** División de una base de datos en partes.
- **Replicación:** Creación de copias de los datos en distintos sitios.

Fin del Tema 1 – Almacenamiento de la Información